

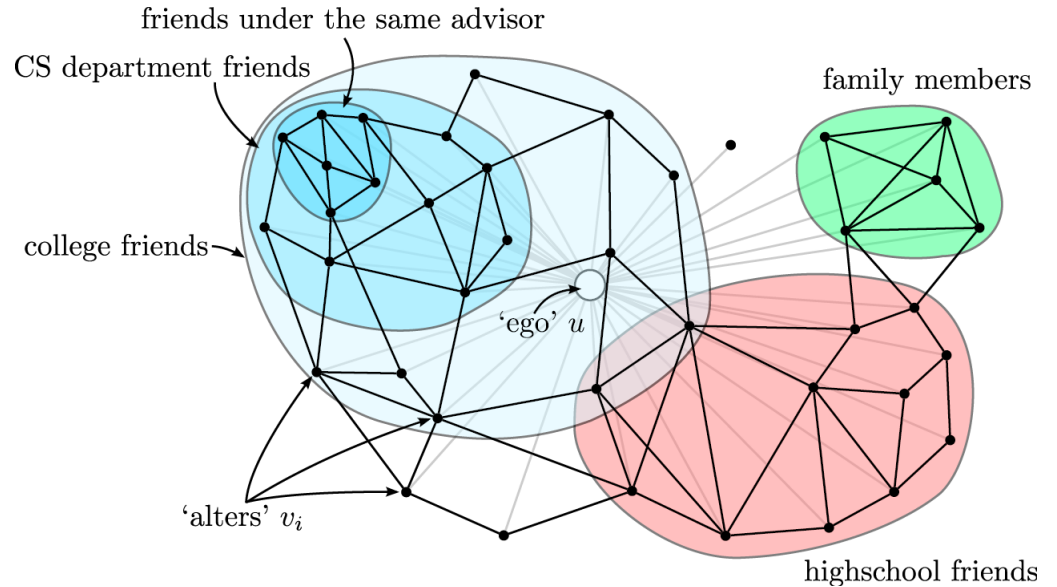
Mining Social Networks

COMPSCI 753

Ruofan Wang

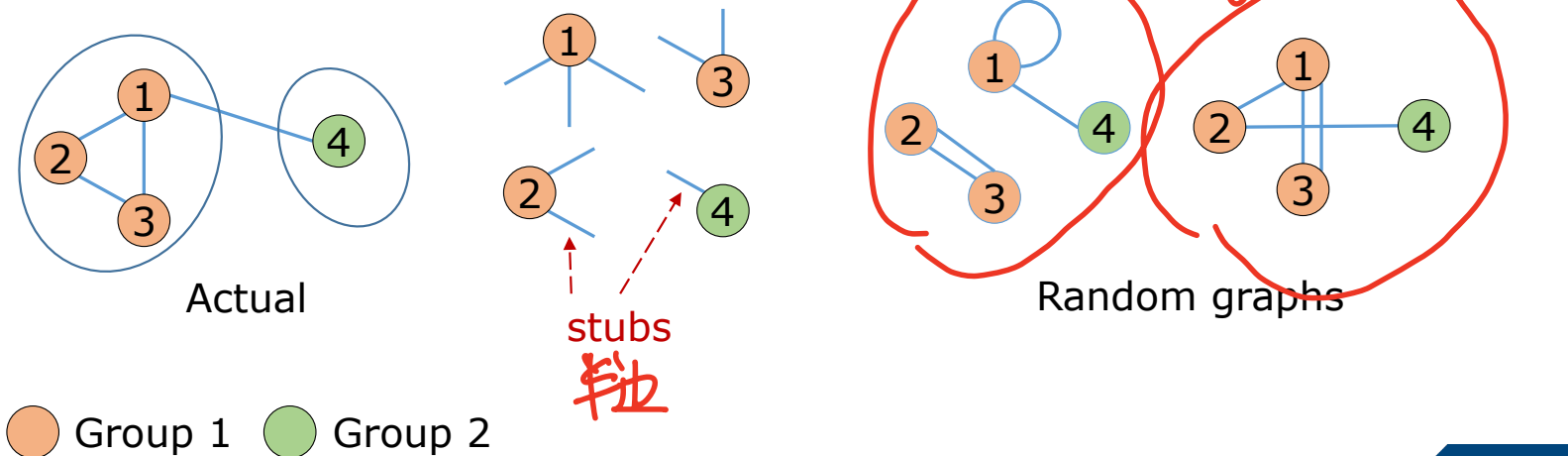
Community detection

- Girvan-Newman algorithm
- Spectral clustering
- **Modularity maximization**



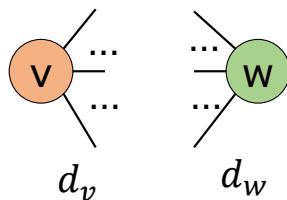
Modularity 模块化.

- The **actual fraction of the edges** that fall within the given groups minus the **expected fraction** if edges were distributed at random
- Note: The node degree and number of edges in the graph should be invariant in random graphs
- Example:



Expected number of edges

- Compute the expected number of edges between any two nodes v and w



- For each stub of v can connect one of the $2m - 1$ stubs, with m the number of edges in the graph

减去连接自身

- Let X_i be the event that the i -th stub of v links to any stub of w . Then

$$E[X_i] = 1 \cdot p(X_i = 1) + 0 \cdot p(X_i = 0) = \frac{d_w}{2m - 1}$$

v 的 stub 连接到 w 的 stub 的概率

w 有 d_w 条边

- The expected number of edges between the two nodes:

$$E \left[\sum_i X_i \right] = \sum_i E[X_i] = \frac{d_v d_w}{2m - 1} \approx \frac{d_v d_w}{2m}$$

所有 v 的 stub 连接到

w 的 stub 的概率

$d_v \left(\frac{d_w}{2m-1} \right)$

Modularity

$$\begin{matrix} x_1 & x_2 \\ \begin{bmatrix} 1 & 0 & \dots \\ 0 & \dots & \dots \end{bmatrix} \end{matrix}$$

- Given adjacency matrix M of the graph, the difference between the **actual** and **expected** number of edges between two nodes v and w

$$M_{vw} - \frac{d_v d_w}{2m}$$

- Suppose we have a set C of communities, the **modularity** is:

$$Q_C = \frac{1}{2m} \sum_{i \in C} \sum_{v,w \in i} (M_{vw} - \frac{d_v d_w}{2m}) = \sum_{i \in C} (e_{ii} - a_i^2)$$

$e_{ij} = \sum_{v \in i, w \in j} \frac{M_{vw}}{2m}$ is the fraction of edges connect community i to j

$a_i = \sum_j e_{i,j}$ is the fraction of edges that has at least one end point in i

e_{ii} : The actual proportion of edges within community i

a_i^2 : If random pairing according to the node degree, the expected proportion of stubs that the community i "self-assigns" to itself.

Difference: how many more edges are there inside the community i than in the random model

Modularity

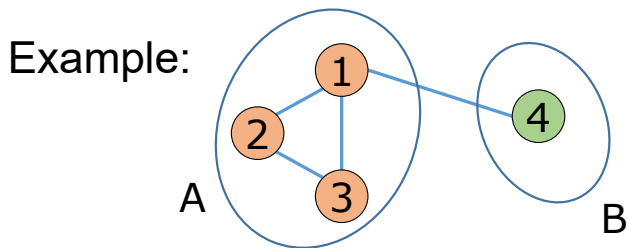
- Given adjacency matrix $\mathbf{M}$ of the graph, the difference between the **actual** and **expected** number of edges between two nodes v and w

$$M_{vw} - \frac{d_v d_w}{2m}$$

- Suppose we have a set C of communities, the **modularity** is:

$$Q_C = \frac{1}{2m} \sum_{i \in C} \sum_{v, w \in i} (M_{vw} - \frac{d_v d_w}{2m}) = \sum_{i \in C} (e_{ii} - a_i^2)$$

$e_{ij} = \sum_{v \in i, w \in j} \frac{M_{vw}}{2m}$ is the fraction of edges connect community i to j
 $a_i = \sum_j e_{i,j}$ is the fraction of edges that has at least one end point in i



	1	2	3	4
1	0	1	1	1
2	1	0	1	0
3	1	1	0	0
4	1	0	0	0

$\mathbf{M}$

Modularity
 Q_C ?

$$e_{AA} = \frac{6}{8}, e_{AB} = \frac{1}{8}, e_{BA} = \frac{1}{8}, e_{BB} = 0$$

$$a_A = \frac{7}{8}, a_B = \frac{1}{8}$$

$$Q_C = \frac{6}{8} - \left(\frac{7}{8}\right)^2 + 0 - \left(\frac{1}{8}\right)^2 = \frac{5}{8}$$

Modularity maximization

- Find a community partition such that the modularity is maximized
- Directly optimizing the modularity is NP-hard – we have exponential different numbers of partitions
- Greedy algorithm (Newman, 2003)
 - Hierarchical clustering
 - Each time we merge two nodes if grouping them results in the largest ΔQ

Greedy algorithm

- Each node is considered as a community
- Merge 2 nodes/communities i and j that results in the largest ΔQ

$$\Delta Q = e_{ij} + e_{ji} - 2a_i a_j = 2(e_{ij} - a_i a_j)$$

- Quick proof:

- Before merging: $Q_{old} = e_{ii} - a_i^2 + e_{jj} - a_j^2$
- After merging: $Q_{new} = e_{ii} + e_{ij} + e_{ji} + e_{jj} - (a_i + a_j)^2$
 $= e_{ii} + e_{ij} + e_{ji} + e_{jj} - a_i^2 - 2a_i a_j - a_j^2$

$$\underbrace{\quad\quad\quad}_{\Delta Q}$$

*Remember our
definition of modularity*

$$Q_C = \sum_{i \in C} (e_{ii} - a_i^2)$$

- Update modularity $Q = Q_{new}$ after merge

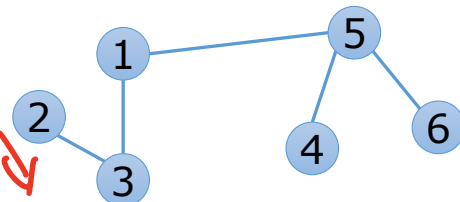
Example

- Calculate $\Delta Q = 2(e_{ij} - a_i a_j)$
 - $\{1,3\} \Rightarrow 2(0.1 - 0.2 * 0.2) = 0.12$
 - $\{2,3\} \Rightarrow 2(0.1 - 0.1 * 0.2) = 0.16$
 - $\{1,5\} \Rightarrow 2(0.1 - 0.2 * 0.3) = 0.08$
 - $\{4,5\} \Rightarrow 2(0.1 - 0.1 * 0.3) = 0.14$
 - $\{5,6\} \Rightarrow 2(0.1 - 0.3 * 0.1) = 0.14$

$a_i =$ 第一行的和.

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \end{bmatrix}$$

标准化



$$e_{ij} = \begin{bmatrix} 0 & 0 & 0.1 & 0 & 0.1 & 0 \\ 0 & 0 & 0.1 & 0 & 0 & 0 \\ 0.1 & 0.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1 & 0 \\ 0.1 & 0 & 0 & 0.1 & 0 & 0.1 \\ 0 & 0 & 0 & 0 & 0.1 & 0 \end{bmatrix}$$

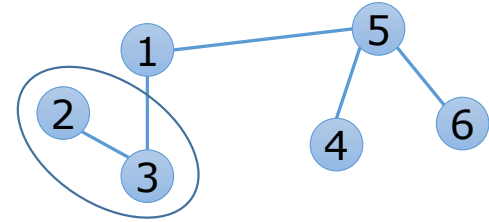
- Max $\Delta Q = 0.16$, combine $\{2,3\}$
- The new $Q = -0.04$

$e_{ij} = \sum_{v \in i, w \in j} \frac{M_{vw}}{2m}$ is the fraction of edges that connect community i to j
 $a_i = \sum_j e_{i,j}$ is the fraction of edges that has at least one end point in i

$$Q_{\text{init}} = \sum_i (e_{ii} - a_i^2) = \sum_i (0 - a_i^2) = -\sum_i a_i^2 = -(0.2^2 + 0.1^2 + 0.2^2 + 0.1^2 + 0.3^2 + 0.1^2) = -0.20.$$

Example

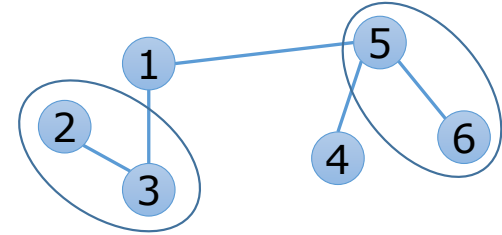
- Calculate $\Delta Q = 2(e_{ij} - a_i a_j)$
 $\{1, \{2,3\}\} \Rightarrow 2(0.1 - 0.2 * 0.3) = 0.08$
 $\{1,5\} \Rightarrow 2(0.1 - 0.2 * 0.3) = 0.08$
 $\{4,5\} \Rightarrow 2(0.1 - 0.1 * 0.3) = 0.14$
 $\{5,6\} \Rightarrow 2(0.1 - 0.3 * 0.1) = 0.14$
- Max $\Delta Q = 0.14$, combine $\{4,5\}$ or $\{5,6\}$.
- Let's say we first combine $\{5,6\}$
- The old $Q = -0.04$
- The new $Q = 0.1$



$$e_{ij}: \begin{bmatrix} 0 & 0.1 & 0 & 0.1 & 0 \\ 0.1 & 0.2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.1 & 0 \\ 0.1 & 0 & 0.1 & 0 & 0.1 \\ 0 & 0 & 0 & 0.1 & 0 \end{bmatrix} \begin{matrix} \leftarrow 1 \\ \leftarrow 2,3 \\ \leftarrow 4 \\ \leftarrow 5 \\ \leftarrow 6 \end{matrix}$$

Example

- Calculate $\Delta Q = 2(e_{ij} - a_i a_j)$
 $\{1, \{2,3\}\} \Rightarrow 2(0.1 - 0.2 * 0.3) = 0.08$
 $\{1, \{5,6\}\} \Rightarrow 2(0.1 - 0.2 * 0.4) = 0.04$
 $\{4, \{5,6\}\} \Rightarrow 2(0.1 - 0.1 * 0.4) = 0.12$

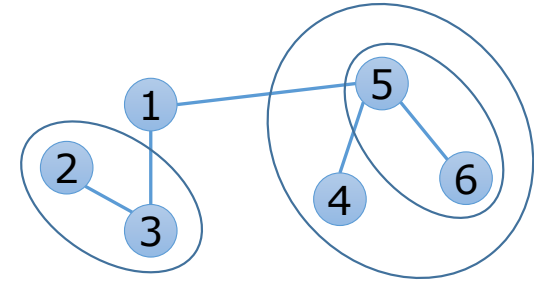


$$e_{ij}: \begin{bmatrix} 0 & 0.1 & 0 & 0.1 \\ 0.1 & 0.2 & 0 & 0 \\ 0 & 0 & 0 & 0.1 \\ 0.1 & 0 & 0.1 & 0.2 \end{bmatrix} \begin{matrix} \leftarrow 1 \\ \leftarrow 2,3 \\ \leftarrow 4 \\ \leftarrow 5,6 \end{matrix}$$

- Max $\Delta Q = 0.12$, combine $\{4,5,6\}$.
- The old $Q = 0.1$
- The new $Q = 0.22$

Example

- Calculate $\Delta Q = 2(e_{ij} - a_i a_j)$
 $\{1, \{2,3\}\} \Rightarrow 2(0.1 - 0.2 * 0.3) = 0.08$
 $\{1, \{4,5,6\}\} \Rightarrow 2(0.1 - 0.2 * 0.5) = 0$

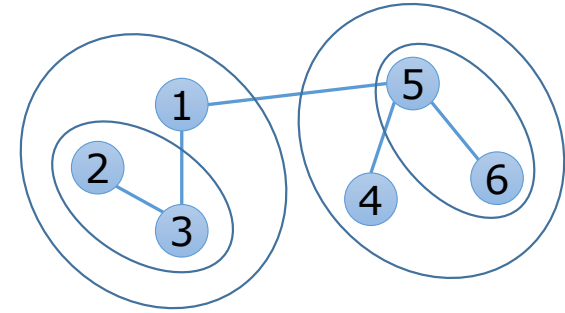


$$e_{ij}: \begin{bmatrix} 0 & 0.1 & 0.1 \\ 0.1 & 0.2 & 0 \\ 0.1 & 0 & 0.4 \end{bmatrix} \begin{array}{l} \leftarrow 1 \\ \leftarrow 2,3 \\ \leftarrow 4,5,6 \end{array}$$

- Max $\Delta Q = 0.08$, combine $\{1,2,3\}$.
- The old $Q = 0.22$
- The new $Q = 0.3$

Example

- Calculate $\Delta Q = 2(e_{ij} - a_i a_j)$
 $\{\{1,2,3\}, \{4,5,6\}\} \Rightarrow 2(0.1 - 0.5 * 0.5) = -0.3$



$$e_{ij}: \begin{bmatrix} 0.4 & 0.1 \\ 0.1 & 0.4 \end{bmatrix}$$

- Max $\Delta Q = -0.3$, combine all in a group.
- The old $Q = 0.3$
- The new $Q = 0$

This algorithm generates a hierarchy of community!

Time complexity - $O((|E| + |V|)|V|)$ ← Number of merges

Compute ΔQ

Choose max ΔQ